Paso 24 — El stop loss para MT5: a cuántos ATR, leído del MAE y sin optimizar

Fecha: 2026-10-08 · Revisado por dos miembros del Council (Feynman, Taleb) el 2026-10-08; correcciones aplicadas. · **Para:** el dueño, que va a fijar los criterios él mismo · **Alcance:** sólo el paso 24: `studies/closing/atrCalculator/` entero, `sqx/variants/stopgrid.py`, el injerto del stop (`sqx/variants/build/stoploss.py`), el capítulo `54-atr-calculator.md` del manual, lo que el autopiloto hace con el paso (`pipeline/autopilot/closing.py`, `traded.py`, `finalists.py`) y su relación con la criba del MAE profundo del paso 8 (`studies/screening/maeDepth/`).

Qué es: el documento de consulta del paso 24, con la misma forma que el de los pasos 15-16 (`criterios-paso-16-spp-2026-10-07`). Explica desde cero qué es el ATR, el MAE y el MFE, cómo se pasa de uno a otro, cómo sale la X del stop y con qué error, qué quiere decir «sin optimizar», y repasa todo lo que el paso escribe y lo que hace con ello el paso 25.

De dónde salen los números. Todos los ejemplos son de la corrida `Trade_NIKKEI225_M30_volatilidadL` del 2026-10-04 (NIKKEI225, M30, largas; lote `AlgoData/atrCalculator/Trade_NIKKEI225_M30_volatilidadL/20261004-203352/`): las mismas diez estrategias del dossier del paso 16. El código del estudio no ha cambiado desde entonces (su último cambio es del 2026-10-04 18:44, anterior a la corrida). Hay dos corridas más recientes del paso (`Test_USDJPY_M30_doubleROCLong` y `Test_USA500_H1_indicesIBS`, 2026-10-06), con tres madres cada una; uso la del Nikkei porque tiene diez y es la que acompaña a los otros dossiers, y cito de las otras sólo lo que guardaron. - **Recalculado por mí con el código del estudio**, desde el export de la pasada 2 (`AlgoData/raw/…/WFC_Build` y `WFC_OOS1`, etiqueta `stopgrid`): todo lo de `build` y `oos1`. Coincide con el `verdict.csv` guardado (diferencia máxima $1{,}4 \cdot 10^{-14}$ en los números; todas las etiquetas iguales). - **`oos2` no lo he recalculado**: lo que aparece de `oos2` está copiado del `verdict.csv` que la corrida escribió. - **«Calculado por mí»** marca lo que el estudio no calcula (qué barras reproducen el MAE, el efecto de camino, el nulo de la transferencia, por qué falla la prueba del ATR). Scripts en la carpeta temporal `scratchpad/paso24/` (`s3_recompute.py`, `s4_mine.py`, `s6_atrproof.py`, `s7_examples.py`, `s5_saved.py`).

Commit descrito: `b2b4c8c`. **Ficheros del paso con cambios sin confirmar:** ninguno en `studies/closing/atrCalculator/`, `studies/screening/maeDepth/`, `sqx/variants/stopgrid.py`, `stoploss.py`, `closing.py`, `traded.py` ni `finalists.py`. Sí los tienen dos piezas que el paso usa: `sqx/variants/build/rewrite.py` (escribe el `.sqx` de cada variante) y `pipeline/autopilot/tailrun.py` (despacha el paso); las dos se tocaron el 2026-10-07, después de la corrida.

Ninguna línea de este documento está congelada. `criteria.yaml:59` lo dice del paso: «Steps 21, 23, 24 annotate only: no rule on purpose».

---

0. En una página

La pregunta del paso. La cadena construye las estrategias sin stop a propósito. El workflow pide uno para operar en fondeo; según el catálogo del proyecto, ni FTMO ni Hantec lo exigen (Hantec lo recomienda), y MT5 admite órdenes a mercado sin stop. Lo que exigen es no pasar la pérdida diaria ni la total, contando el

flotante. ¿A qué distancia se pone, en múltiplos de ATR, sin convertir esa distancia en un parámetro más que ajustar?

Lo que hay cargado hoy. El estudio no elige: calcula cuatro X por estrategia (percentiles 80, 85, 90 y 95 del MAE/ATR de las ganadoras del `build`) y las enseña lado a lado. **Quien elige es una línea del ledger**, `portfolio.funded.stop_percentile = 90` (dueño, 2026-09-30), que lee el paso 25: la versión que sigue hacia MT5 es, para todas las estrategias, la del percentil 90. No hay ninguna regla en `criteria.yaml`, ningún hecho en `facts.py` y ningún corte: del paso 24 no sale nadie.

Lo que sale en la corrida real. Diez estrategias, X del p90 entre 1,18 y 4,59 ATR (295 a 1.148 USD por operación en estas cuentas de 100.000). Las diez pasan al paso 25 con su stop. Con ese stop, la suma del neto de las diez baja de 214.787 a 183.190 USD en `build` (−15 %) y de 99.724 a 67.559 en `oos1` (−32 %); en `oos1` ocho de diez ganan menos y dos que ganaban pasan a perder. La peor operación se queda en un tercio de lo que era.

El −15 % de `build` es casi entero de dos estrategias (`16.5.352` −16.999 y `18.5.355` −12.828 USD, el 94 % de los −31.597 USD de `build`); en las otras ocho el stop apenas cuesta en la ventana donde se leyó la X (mediana −1.089 USD por estrategia, y cinco de diez ganan más con stop). Fuera cuesta en ocho de diez (mediana −4.414 USD) y las dos peores son el 40 % del total. La cifra para decidir es la de `oos1` (calculado por el revisor, de la tabla de 3.6).

Todo lo medido aquí es de diez estrategias largas del Nikkei en M30. Nada está comprobado en cortas (cuyo flotante va contra el ask) ni en otro activo.

Las catorce cosas que conviene saber antes de leer el resto:

1. **El stop no es neutro.** El manual dice que «lo esperable es que salte poco y cambie poco». Medido: al p90 salta en el 23-41 % de las operaciones del `build` (mediana 36 %) y en `oos1` se lleva un tercio del neto. Mata el 10 % de las ganadoras, como promete la regla, y esas ganadoras son las grandes (−152.489 USD en `build`, el 71 % del neto original); cortar perdedoras compensa +98.701; las operaciones nuevas aportan +14.241 y las desaparecidas +7.950 (calculado por el revisor, descomposición del cambio de neto al p90, suma de las diez). Además cambia qué operaciones existen (punto 2).
2. **Con stop la estrategia es otra.** Al cerrar antes, queda libre para señales que la original no pudo tomar. Al p90, en `build`: de 1.891 operaciones originales desaparecen 30 y aparecen 196 nuevas (un 10 % más). Esas operaciones nuevas no han pasado por ninguno de los pasos 8 a 20.
3. **Nada vuelve a juzgar a la versión con stop.** El autopiloto sólo le pasa el paso 25 (edge por coste, que lee sólo `oos1`: el `build` se archiva y no se juzga; `studies/readings/edgeCost/costs.py:64`). La que llega a MT5 no es la que superó la cadena.
4. **El stop no pone techo a la pérdida.** El Nikkei abre con hueco. En `build`, 43 de los 688 stops de las diez versiones que se operan (6,2 %) se ejecutaron en una apertura ya por debajo del nivel, en 37 días distintos: mediana 0,9 ATR más allá, máximo 12 ATR; en `oos1`, 10 de 450 (2,2 %), máximo 2,2 ATR (calculado por el revisor sobre las variantes del p90; el 5,5 % de 3.5 es de toda la rejilla de 200 variantes). La peor operación con stop al p90 es entre 1,15 y 3,6 veces la pérdida nominal del stop (mediana 1,9). La peor operación observada no es un techo: en `build` la fijan tres operaciones de `16.5.352`, y el backtest a M30 no puede ver ningún exceso dentro de la vela (todo stop sin hueco se llena exacto en el nivel). Lo medido es un mínimo de lo que el stop se pasa.

5. **La «prueba del ATR» dice «no» en ocho de diez y nadie lo lee.** No es que el ATR esté mal: quitando el deslizamiento y los huecos, la distancia del stop es X·ATR con cuatro decimales. Falla porque su tolerancia (0,02) se calibró en el oro y porque el deslizamiento es fijo y hay huecos, sobre todo en `build`. El manual dice «si alguna dice no, lo demás no vale»; el autopiloto sigue igual. En las otras cuatro corridas del Nikkei falla en diez de diez.
6. **La X tiene un error grande y la etiqueta «se transfiere» no distingue nada.** Con 77-152 ganadoras, el intervalo del p90 mide entre el 26 % y el 71 % de la propia X. Y con estas muestras, dos ventanas con la misma distribución darían «no se transfiere» entre el 22 % y el 44 % de las veces (calculado por mí); se observan 5 de 10.
7. **La rejilla de estabilidad no la lee nadie.** Cuesta 20 variantes por estrategia y una mirada más a `oos2`, produce la etiqueta «meseta/borde», y el paso 25 toma siempre el centro del p90 diga lo que diga la etiqueta. En la corrida sólo 3 de 10 son «meseta» en `build` al p90.
8. **El «DD máximo» de la tabla de coste y de la puntuación es sobre cierres, no sobre flotante** (`stability.py:17`). En fondeo manda el flotante. Ese drawdown, además, sube con el stop en 5 de 10 estrategias en `build` y 6 de 10 en `oos1`.
9. **El coste del stop viene de las ganadoras grandes que mata, no de cortar perdedoras.** Cortar perdedoras ayuda en neto; lo que cuesta es el 10 % (13 % en `oos1`) de ganadoras que se lleva, que son las que van hondo y luego rinden (5.15).
10. **«Las cuentas de fondeo exigen un stop» es falso según el catálogo del proyecto** (`ftmo.yaml:164`: «No stop loss required»; `hantec.yaml:258`: «not mandatory, strongly recommended»). El stop es una decisión de riesgo, no un requisito de la firma (5.16).
11. **No hay ninguna cifra de las diez juntas.** Son largas del mismo índice, llegan a estar hasta 8 abiertas a la vez, y el peor día por cierres con stop es −5.311 USD en `build` y −4.303 en `oos1` (calculado por el revisor, por cierres y sin flotante; 5.17).
12. **Los 250 USD por ATR salen del money management** (`assets/_build.yaml:60-63`: `Amount` 1000 ÷ `ATRMult` 4), y con X > 4 la pérdida nominal del stop supera los 1.000 USD con que se dimensiona (5.18).
13. **«La peor operación observada» es un suelo, no un techo**: en `build` la fijan tres operaciones de una sola estrategia (5.19).
14. **El percentil 90 no se fijó «antes de ver ningún resultado con stop»**: se fijó el 2026-09-30, después de las corridas de desarrollo del oro y de USDJPY, cuya tabla de coste por percentil ya se había visto (5.20).

---

1. Qué es el paso y cómo decide

1.1 Dónde está

Familia «cierre» (`studies/closing/`), después de la exposición (paso 22) y la estructura (23) y antes del edge por coste de la versión que se opera (25) y de la verificación en MT5 (26). Encargo del dueño del 2026-09-25/26, con una frase que marca el diseño: *«No quiero que sea una optimización ni nada»*.

1.2 La mecánica: dos pasadas de SQX con Python en medio

La lanza `pipeline/autopilot/closing.py:101` (`stop`), sobre **todas** las madres que quedan y dentro del proyecto de la propia corrida, usando sus tres tareas WFC (`WFC 1 IS`, `WFC 2 OOS1`, `WFC 3 OOS2`), que son tres retests de la misma estrategia sobre `build`, `oos1` y `oos2`.

pasada	qué fabrica `sqx.variants.stopgrid`	qué hace SQX	qué hace el estudio después
1	por madre, 2 ficheros: la original sin tocar y una «sonda» con stop a 1.000 ATR	los retestea en las tres ventanas	lee el MAE de la original, calcula las X y escribe `stopgrid.csv`
2	por madre, 22 ficheros: los dos anteriores y 20 con stop (4 percentiles × 5 valores alrededor de cada X)	los retestea en las tres ventanas	repite la lectura y añade las pruebas, el coste y la forma

En la corrida real: 10 madres → 20 ficheros en la pasada 1 y 220 en la 2; unos ocho minutos en total (de 20:33 a 20:41). Si una pasada falla, `closing.py:133-138` mueve el lote a medias a `autopilot/<proyecto>/<corrida>/paso-24-abortado` para que el paso 25 no lo tome por terminado.

Las tareas WFC llevan además once mercados adicionales. El estudio los descarta (`inputs.py:111-112`): de las 555.658 operaciones exportadas de `WFC_Build`, 45.608 (el 8 %) son del Nikkei. El manual pide configurar las patas con `--no-markets` para este paso; el autopiloto no lo hace.

1.3 Qué datos lee

- **Las operaciones** de cada variante, una fila por operación, con el `MAE ($)` y el `MFE ($)` que escribe SQX. El tramo de cada operación (`build`, `oos1`, `oos2`) se asigna por su fecha de apertura contra `assets/_policy.yaml` (`inputs.py:113-116`). En el Nikkei: `build` 2011-09-19 a 2019, `oos1` 2020-2023, `oos2` 2024 a 2026-09-30.
- **Las barras M30** del activo (remuestreadas desde M1), para calcular el ATR.
- **El `point_value`** del activo (0,063137 USD por punto y lote en el Nikkei), para pasar dólares a precio.

Lee `oos2`. Está autorizado: `ATRStop` figura en `reserved_for` de `assets/_policy.yaml:48` (dueño, 2026-09-26). Cada pasada pide permiso a la puerta (`report.py:45`) y firma una fila del ledger por tramo (`spend.py:24-50`). La corrida real dejó 6 filas (2 pasadas × 3 tramos): **dos miradas a `oos2` por corrida**. Las filas llevan `n_in = n_out` y ninguna puntuación, porque el paso no descarta a nadie.

1.4 Quién decide

quién	qué decide	dónde
el estudio	nada: veredicto `info`, «4 X, sin elegir»	`one.py:87-94`
el autopiloto, paso 24	nada: sin reglas, sin hechos, sin corte	`criteria.yaml:59`, `tail.py:16`
el ledger	**el percentil con el que se opera: 90**, el mismo para todas	`ledger/thresholds.yaml:1133-1138`
el paso 25	toma, de cada madre, la variante «percentil 90, paso 0 de la rejilla»	`traded.py:47`

1.5 Las estrategias que nos acompañan

Las diez de `Trade_NIKKEI225_M30_volatilidadL`. Tabla de sus X (recalculada; `*` = marcada «poco fiable», intervalo más ancho que media X):

estrategia	ops `build`	ganadoras	X p80	X p85	X p90 [intervalo 95 %]	X p95
10.11.484	154	83	1,65*	1,95	**2,21** [1,79 – 2,71]	2,69
13.3.390	143	77	2,60	2,99	**3,34*** [2,71 – 5,09]	5,07*
14.3.479	189	80	0,89	1,05	**1,18*** [0,94 – 1,64]	1,64*
15.9.352	224	115	1,72	1,91	**2,27** [1,85 – 2,58]	2,65*
16.5.352	304	152	2,97	3,70	**4,59** [3,65 – 5,65]	5,88
18.5.355	161	98	3,32	3,53	**4,20** [3,48 – 5,45]	5,45
19.2.363	161	87	1,05	1,16	**1,26** [1,13 – 1,54]	1,48*
3.11.426	135	78	1,87	2,01	**2,35** [1,92 – 3,03]	2,95*
3.9.284	222	109	1,90*	2,28*	**2,55** [2,16 – 3,02]	3,21*
4.13.338	198	110	2,23	2,38	**2,75** [2,36 – 3,09]	3,24

El `build` del Nikkei dura 8,3 años: entre 16 y 37 operaciones al año. Nueve de las diez están por debajo del suelo de trabajo de 35; eso no es materia de este paso, pero es la razón de que las muestras de ganadoras sean tan cortas.

---

2. Fundamentos de este paso

2.1 El ATR: cuánto se mueve el precio en una vela

El rango verdadero (true range, TR) de una vela es lo que el precio recorrió en ella, contando el hueco con el cierre anterior. Es el mayor de tres números: máximo − mínimo; |máximo − cierre anterior|; |mínimo − cierre anterior| (`engines/market/atr.py:9-22`).

El ATR (average true range) es una media de los TR de las últimas velas. SQX no usa la media simple, sino la de Wilder, que da a la vela nueva un peso de 1/n y al valor anterior el resto:

> ATR de hoy = ((n − 1) × ATR de ayer + TR de hoy) / n

con n creciendo 1, 2, 3… hasta el periodo, y fijo desde ahí (`atr.py:44-47`). Ejemplo a mano con periodo 3:

vela	máx	mín	cierre	TR	n	ATR
1	100	98	99	2	1	2
2	101	99	100	max(2; 2; 0) = 2	2	(1·2 + 2)/2 = 2
3	104	102	103	max(2; |104−100| = 4; 2) = 4	3	(2·2 + 4)/3 = 2,67
4	104	102	103	2	3	(2·2,67 + 2)/3 = 2,44

La vela 3 abre con hueco: su rango interno es 2, pero el TR es 4 porque cuenta el salto desde el cierre anterior.

Con qué periodo y qué temporalidad. Periodo **20**, siempre (`config.yaml:9`, y escrito como literal en el fichero de la estrategia: `stoploss.py:6`). Temporalidad: **la de la estrategia** (M30 aquí; `load.py:48`). Un ATR(20) en M30 resume las últimas diez horas de mercado.

De qué vela. De la última vela **cerrada antes** de la vela de entrada (`excursion.py:12-23`). Es lo que SQX llama desplazamiento 1 y es el dato que la estrategia conoce cuando entra.

Órdenes de magnitud en el Nikkei (calculado por mí, entradas del `build`): ATR(20) de M30 mediano 38 puntos (0,21 % del precio), entre 21 (percentil 5) y 87 (percentil 95). Un stop a 2,2 ATR son unos 85 puntos.

Por qué en ATR y no en puntos. Un stop de 80 puntos es enorme en un día tranquilo y diminuto en uno agitado. En múltiplos de ATR la distancia se adapta sola a la volatilidad del momento.

Lo que vale un ATR en dinero, en esta corrida (calculado por mí): `tamaño × point_value × ATR` da **250 USD en todas las operaciones de las diez estrategias** (mínimo 249,9, máximo 250,0). Es decir, el tamaño de cada operación está fijado para que un ATR valga 250 USD. Sale de `assets/_build.yaml:60-63` (money management `ATRRiskBasedSizingFixedRisk`, `Amount` 1000, `ATRMult` 4, `ATRPeriod` 20): 1.000 ÷ 4 = 250 USD por ATR; el tamaño se fija para arriesgar 1.000 USD a 4 ATR, sin relación con el stop y con independencia del capital (`knowhow/sqx-format/money-management-atr-fixed-risk.md`). Consecuencia práctica: un stop a X ATR es una pérdida nominal de X × 250 USD (2,21 ATR → 553 USD, el 0,55 % de los 100.000 de capital inicial). El lote es por tanto inverso al ATR: el stop nominal vale lo mismo en dinero con cualquier volatilidad, pero una entrada tras diez horas quietas lleva más lote y un hueco de apertura no se encoge con ese ATR (5.18). La cartera recalcula el tamaño con la X: `factor = ATRMult / (Amount · X)` (`portfolio/common/construct/inputs/risk.py:49-50`).

2.2 MAE y MFE: lo peor y lo mejor que llegó a ir una operación mientras estuvo abierta

- **MAE** (maximum adverse excursion): la mayor pérdida **flotante** que tuvo la operación entre que abrió y que cerró. En una compra, la distancia del precio de entrada al punto más bajo que tocó.
- **MFE** (maximum favourable excursion): la mayor ganancia flotante.

Una operación puede cerrar ganando 13 USD y haber ido 843 en contra por el camino. El cierre no lo cuenta; el MAE sí. Por eso en fondeo, donde la cuenta se pierde por el flotante, el MAE importa tanto como el resultado.

Cómo los exporta SQX. Dos columnas por operación, `MAE ($)` (negativa) y `MFE ($)`, en dólares de la cuenta. La regla, medida en `knowhow/export/mae-mfe-from-m1.md`: se miran las mechas de las barras desde la de entrada hasta la **anterior** a la de salida, más la apertura de la barra de salida, contra el precio de ejecución.

Con qué precisión, y qué cambia. Las tareas WFC corren a `testPrecision="1"`, que en SQX es «sólo el timeframe elegido»: el backtest ve las velas M30, no los minutos ni los ticks (`assets/_build.yaml:98-100`, dueño 2026-10-02; lo he leído en el `project.cfx` de la corrida, `Retest-Task16/17/18.xml`; los once mercados adicionales de esa tarea corren a `testPrecision="2"`, lo que agrava el coste de 1.2). ¿Cambia eso el MAE? **En estas estrategias, no** (calculado por mí, 1.891 operaciones originales del `build`):

reconstrucción del MAE	coincide con el exportado (a medio punto)
mínimos de las velas **M30**, de la de entrada a la anterior a la salida, más apertura de la vela de salida y precio de cierre	100,0 %
lo mismo con las velas **M1**	100,0 %

La razón es sencilla: estas estrategias entran y salen en aperturas de vela M30. Mientras la operación vive velas enteras, el mínimo de una vela M30 **es** el mínimo de sus treinta velas M1, y el de una vela M1 es el tick más bajo que la formó. La X del p90 sale idéntica con las tres. (La prueba de que las patas corren a M30 no sale de los datos, que no distinguen M30 de M1 aquí, sino de `assets/_build.yaml:98-100` y del `project.cfx`.)

Dónde **sí** cambiaría:

- **Entradas o salidas dentro de la vela** (órdenes stop o limit, un objetivo de beneficio): a M30 SQX no sabe si el mínimo de la vela vino antes o después de la entrada. Hoy las órdenes son a mercado.
- **Ticks reales con spread variable**: las barras son de precio *bid*. Una compra flota contra el bid, así que la barra dice la verdad. Una venta flota contra el *ask* (bid + spread): con el spread real, que se abre en noticias y en el cambio de día, su MAE sería más hondo que el de las barras. Las diez de aquí son largas; el caso corto no está comprobado ni en la tarjeta de knowhow.
- **La versión con stop**: a M30 el stop se da por tocado si el mínimo de la vela llega al nivel, y la hora de cierre se sella en la **apertura de esa vela**, no en el minuto real (hasta 29 minutos después). Para una regla de pérdida diaria intradía, esa hora no vale.

2.3 De dólares a múltiplos de ATR

Tres divisiones (`core/trades.py:57-58` , `excursion.py:42-46`):

> MAE en precio = |MAE ($)| / (tamaño × point_value) MAE en ATR = MAE en precio / ATR(20) de la vela cerrada antes de la entrada resultado en ATR = beneficio neto / (tamaño × point_value) / ATR

Se usa el tamaño **de cada operación**, porque varía de una a otra.

Ejemplo real (`Strategy 10.11.484` , compra del 2012-08-08 03:30 a 8.933,30, 163,42 lotes, cerrada el día 9 a las 05:00 en 8.935,76 con +13,14 USD):

- `MAE ($)` = −843,07. Un punto vale 163,42 × 0,063137 = 10,318 USD. MAE en precio = 843,07 / 10,318 = **81,7 puntos**.
- ATR(20) de la vela anterior = 24,23 puntos. MAE en ATR = 81,7 / 24,23 = **3,37**.
- Resultado en ATR = 13,14 / 10,318 / 24,23 = +0,05.

Es una ganadora (+13 USD) que estuvo 3,37 ATR en contra. Con el stop del p90 (2,21 ATR = 53,6 puntos, nivel 8.879,73) SQX la cierra en 8.878,68 con **−563,56 USD**. Es una de las nueve ganadoras que ese stop mata en esta estrategia.

Qué es «ganadora»: beneficio neto de costes mayor que cero (`excursion.py:46`).

2.4 Cómo sale la X: un percentil de las ganadoras

Percentil. El percentil 90 de una lista es el valor que deja por debajo al 90 % de sus elementos. Ejemplo con diez ganadoras cuyo MAE en ATR, ordenado, es:

> 0,2 · 0,3 · 0,5 · 0,6 · 0,8 · 1,0 · 1,2 · 1,5 · 2,0 · 3,5

numpy coloca el percentil p en la posición p × (n − 1) contando desde cero, e interpola. Para p = 0,90: posición 8,1, entre el noveno valor (2,0) y el décimo (3,5): 2,0 + 0,1 × 1,5 = **2,15**. Un stop a 2,15 ATR habría dejado intactas nueve de las diez y matado a la de 3,5.

La regla del paso (`threshold.py:44`): X = percentil p del MAE/ATR de **las ganadoras del `build`**, para p = 80, 85, 90 y 95 (`config.yaml:10`).

Por qué sólo las ganadoras. El stop es un colchón: debe dejar respirar a las operaciones que acaban bien. A las perdedoras da igual cortarlas, y de hecho van mucho más hondo. En `10.11.484`: MAE mediano de las ganadoras 0,82 ATR; de las perdedoras 2,82. Por eso una X que deja pasar al 90 % de las ganadoras corta bastante más del 10 % de **todas** las operaciones: en esa estrategia X = 2,21 es el percentil 63 del conjunto, es decir, el 37 % de sus operaciones llega ahí. La cuenta general es fracción = w × 10 % + (1 − w) × q, con w la parte de ganadoras y q la fracción de perdedoras que llega a X (35-70 % en las diez; aquí 0,54 × 0,108 + 0,46 × 0,676 = 37 %; calculado por el revisor). Es además forzoso: una perdedora que pierde más de X ATR ha tenido MAE ≥ X. (Y el «% de paradas» divide por las operaciones de la variante, nuevas incluidas: 2.057 y no 1.891.)

Por qué sólo el `build`. Para que la X no vea los tramos con los que luego se comprueba. `oos1` y `oos2` sólo dicen si se transfiere (2.9). Si el export no trae `build`, el estudio se niega (`one.py:33-37`).

En la estrategia real (83 ganadoras): posición 0,9 × 82 = 73,8, entre la 74.ª (2,20) y la 75.ª (2,21) → X = **2,21**. Las que quedan por encima son nueve: 2,35 · 2,36 · 2,47 · 2,71 · 2,92 · 2,95 · 3,37 · 5,56 (y la 75.ª, a ras).

2.5 El error de muestreo de un percentil

La X no es un número exacto: sale de 83 operaciones, y con otras 83 de la misma estrategia saldría otra. Dos maneras de ver cuánto.

A mano, con la binomial. Si la X «verdadera» deja por debajo al 90 % de las ganadoras, el número de las 83 que caen por debajo es una cuenta de 83 monedas cargadas al 90 %: media 74,7, desviación √(83 × 0,9 × 0,1) = 2,7. Con dos desviaciones a cada lado: entre 69 y 80 (la binomial exacta da 69 y 80). La X verdadera puede estar, con un 95 % de confianza, entre la ganadora n.º 69 y la n.º 80 de la lista ordenada: entre **1,83 y 2,92** (con la 81, 2,95). La estimación (2,21) depende de dónde caen exactamente unas doce operaciones.

Lo que hace el estudio: bootstrap (`threshold.py:9-28`). Saca 9.999 muestras de 83 ganadoras **con reemplazo** de las propias 83, calcula el percentil en cada una y se queda con el 95 % central de esos 9.999 valores. Para esta estrategia: **1,79 a 2,71**, parecido a la cuenta a mano. Si el ancho del intervalo pasa de la mitad de X (`bootstrap.wide: 0.5`), la X se marca «poco fiable» (`threshold.py:48`) y sale un aviso; no bloquea nada.

Lo que da en la corrida (ancho del intervalo como parte de X, recalculado):

percentil	ancho relativo, de la más estrecha a la más ancha	«poco fiable»
80	23 % – 62 %	2 de 10
85	24 % – 51 %	1 de 10
90	26 % – 71 %	2 de 10
95	27 % – 132 %	6 de 10

Dos lecturas. Una: cuanto más alto el percentil, menos operaciones lo deciden (al p95 con 80 ganadoras son las cuatro más extremas) y más baila. Dos: la marca «poco fiable» salta en 0,50, pero una X con un intervalo del 47 % no es mucho más de fiar que una del 51 %. El bootstrap, además, supone que las operaciones son independientes entre sí; las de una misma racha de mercado no lo son, y entonces el intervalo real es más ancho que el que sale.

2.6 Qué quiere decir «sin optimizar»

Optimizar sería: probar muchas distancias de stop, mirar el resultado de cada una (neto, PF, drawdown) y quedarse con la mejor.

Por qué sobreajusta. El resultado con cada X tiene una parte real y una parte de suerte de la muestra. Al elegir la mejor se elige también la que más suerte tuvo. Ejemplo: veinte candidatas que en realidad valen lo mismo, cada una con un ruido de desviación σ; la mejor de las veinte sale, de media, 1,9 σ por encima de su valor verdadero. Ese exceso es puro ruido y no se repite fuera. (Con X vecinas el efecto es menor, porque comparten casi todas las operaciones, pero no desaparece.)

Qué hace el paso en su lugar. La regla —«el percentil p del MAE de las ganadoras»— está escrita **antes** de ver ningún resultado con stop de esta corrida, es **la misma para todas** las estrategias y no mira ni el neto ni el PF de la versión con stop: sólo la distribución del MAE de la versión sin stop. No hay ningún `max()` sobre resultados en el código: `grid.py` genera las X por fórmula, `stability.py` etiqueta la forma, y `traded.py:47` toma el centro del percentil del ledger.

Lo que sigue siendo libre. La regla tiene un parámetro: el percentil. El 90 se fijó el 2026-09-30, después de las corridas de desarrollo (oro 2026-09-26, USDJPY 2026-09-28/30), cuya tabla de coste por percentil ya se había visto (el capítulo del manual del 2026-09-29 la trae para el oro), y antes de la corrida del Nikkei (2026-10-04) y de cualquier cartera. No consta si el 90 se eligió mirándolas. Es una regla previa a esta corrida, no ciega al coste del stop (5.20). Pero el informe enseña cuatro percentiles con su coste lado a lado; **si el percentil se cambia después de mirar esa tabla, es una optimización de cuatro candidatas hecha a ojo**, y el ledger no la contaría (la fila del paso 24 lleva `n_in = n_out`).

Qué habría pasado optimizando, en la corrida real (calculado por mí, `build` y `oos1`; es una ilustración, no un ensayo: diez estrategias y veinte X muy correlacionadas). De las veinte X de la rejilla de cada madre, tomo la de mayor neto en `build` y miro su neto en `oos1` frente al de la X del p90:

- En `build` gana siempre (por construcción): +44.192 USD entre las diez.
- En `oos1`: mejor que el p90 en 7 de 10 (+18.010 USD en total), peor en 3. En esas tres la elegida queda en los puestos 13, 19 y 19 de 20 en `oos1`.
- La X «mejor» es más floja (más lejos) que la del p90 en 8 de 10.

- El orden de las veinte X por neto en `build` se parece al de `oos1` de forma muy desigual: correlación de rangos entre 0,03 y 0,93 según la estrategia (mediana 0,51).

Lo que eso enseña no es que optimizar funcione, sino por qué: **en estas estrategias el stop cuesta dinero, y cuanto más lejos, menos cuesta**. El coste de un stop más cercano es matar más ganadoras grandes, no cortar perdedoras (5.15). «Maximizar el neto» tiende a elegir el stop más flojo, que es acercarse a no tener stop. Y si el percentil se moviera mirando la columna de `oos1`, el paso 25 (que sólo lee `oos1`) lo confirmaría sobre ese mismo tramo. El stop no está para ganar más: está para acotar la peor operación. Un criterio de optimización sobre el neto lo destruye, y uno sobre el drawdown de cierres tampoco lo recoge (5.6).

2.7 El sesgo de leer el MAE en operaciones que vivieron sin stop

El MAE se lee de las operaciones de la versión **sin** stop. Poner el stop cambia tres cosas que esa lectura no ve.

(a) Cierra perdedoras que recuperaban parte. Una perdedora que fue 4 ATR en contra y cerró en −1 ATR pasa a cerrar en −2,2. La regla sólo protege a las ganadoras.

(b) Cambia qué operaciones existen. Ejemplo: la estrategia entra en la vela 1 y sale en la 10; en la vela 6 hay otra señal, que ignora porque ya está dentro. Con stop, la primera operación se cierra en la vela 4; en la 6 la estrategia está libre y **entra**. Esa operación no existía. Y al revés: una operación nueva puede seguir abierta cuando llegaba una original, que entonces desaparece.

(c) Las operaciones nuevas no tienen MAE medido. La X no las conoce.

Medido en la corrida (calculado por mí; stop del p90, centro de la rejilla, suma de las diez):

	`build`	`oos1`
operaciones originales	1.891	1.162
la lectura de Python predice que el stop cierra (MAE ≥ X)	618	401
de las originales que siguen existiendo, SQX cierra por stop	605	393
discrepancias entre Python y SQX en las que siguen existiendo	0	1
originales que **desaparecen**	30	19
operaciones **nuevas**	196 (10,4 %)	131 (11,3 %)
operaciones con stop	2.057	1.274
ganadoras del original que el stop mata	102 de 989 (10,3 %)	80 de 615 (13,0 %)
neto de las nuevas	+14.241 USD	+8.867 USD

Tres conclusiones. **Uno**, sobre las operaciones que siguen existiendo, la lectura de Python y el retest de SQX coinciden operación por operación: el MAE exportado predice sin error cuál salta. **Dos**, la promesa «deja intactas al 90 % de las ganadoras» se cumple en `build` (10,3 % muertas) y casi en `oos1` (13,0 %). **Tres**, el efecto de camino no es pequeño y es muy desigual: `18.5.355` gana 1 operación nueva; `15.9.352` tiene 73 nuevas y pierde 8 (de 224 pasa a 289). En esa última, la estrategia con stop comparte con la original tres de cada cuatro operaciones.

Hay un cuarto sesgo, **no medido**: las estrategias se eligieron por su resultado en `build`. Es razonable esperar que sus ganadoras del `build` tengan un MAE algo menor que el que tendrán fuera, y que la X salga corta. En la corrida, las ganadoras de `oos1` necesitaron más aire que las del `build` en 9 de 10 estrategias (cociente de medianas mayor que 1; `10.11.484` queda en 1,002, que la tabla de 2.9 redondea a 1,00). Como moneda al aire saldría el 2 % de las veces (a dos colas, calculado por el revisor), pero las diez comparten mercado y periodo (no son independientes) y el dato sigue sin demostrar nada por sí solo. Juntando las diez, matan 80 ganadoras de 615 en `oos1` donde la regla promete 61: entre 1,8 y 2,5 desviaciones según se cuente el error de la X (calculado por el revisor con una binomial sobre 989 ganadoras en `build` y 615 en `oos1`; supone operaciones independientes, que no lo son, así que es un techo de la evidencia). Es un indicio de que la X del `build` queda corta fuera, no una prueba.

2.8 El punto sin retorno

Otra lectura del mismo MAE, que no produce la X sino una etiqueta (`noreturn.py:9-36`). Para cada distancia x de 0,25 en 0,25 ATR hasta 8: de **todas** las operaciones del `build` (ganadoras y perdedoras) cuyo MAE llegó a x, cuántas acabaron ganando (`recovered`) y cuál fue su resultado medio final en ATR (`mean_final`).

La zona es **«sin retorno»** cuando se cumplen dos cosas (`noreturn.py:32-33`): como mucho el 10 % se recuperó, **y** su resultado medio es peor que −x (cortar ahí habría salido más barato que dejarlas). Si no, **«ruido»**. Con menos de 10 operaciones llegando a x, «pocas operaciones».

Ejemplo real, `10.11.484`:

x (ATR)	llegaron	acabaron ganando	resultado medio final	zona
1,00	100	33 %	−1,01	ruido
2,00	62	18 %	−2,24	ruido
2,50	44	11 %	−3,08	ruido
2,75	43	9 %	−3,17	sin retorno
3,00	35	6 %	−3,67	sin retorno

En esta estrategia el punto sin retorno empieza en 2,75 ATR. Sus cuatro X (1,65 a 2,69) caen por debajo: «ruido» las cuatro. **En la corrida las cuarenta X (10 × 4) son «ruido».** No es casualidad: en X = p90 se recupera el 10 % de las ganadoras, pero «recuperado» en la curva es la parte de ganadoras sobre TODAS las que llegan; para que fuera ≤ 10 % harían falta perdedoras ≥ 0,9 × ganadoras llegando a X, cosa imposible si hay más de 1,11 ganadoras por perdedora (en 6 de 10 madres lo es; en las otras 4 haría falta que llegara casi toda perdedora, y llega el 35-67 %; calculado por el revisor). Por eso, en la práctica, el p90 nunca cae en «sin retorno».

2.9 La transferencia: ¿vale fuera la X de dentro?

Percentil efectivo (`transfer.py:27`): qué porcentaje de las ganadoras de `oos1` (o `oos2`) tiene un MAE menor o igual que la X del `build`. Si la X del p90 resulta ser el percentil 75 en `oos1`, allí el stop mata a una de cada cuatro ganadoras en vez de a una de cada diez.

«Se transfiere» (`transfer.py:30`) si el efectivo queda a 5 puntos o menos del pretendido (`transfer.tolerance: 5.0`) y la ventana tiene al menos 30 ganadoras (`min_winners: 30`).

Tres números más, de la distribución entera (`transfer.py:35-55`): - **D de Kolmogorov-Smirnov**: se dibujan las dos curvas acumuladas (qué fracción de ganadoras tiene MAE ≤ x, para cada x) y D es la mayor separación vertical entre ellas, de 0 a 1. Su p dice con qué frecuencia saldría una D así de grande si las dos muestras vinieran de la misma distribución. - **Anderson-Darling**: otra comparación de dos muestras que pesa más las colas. scipy recorta su p entre 0,001 y 0,25. - **Cociente de medianas**: mediana del MAE de las ganadoras de fuera entre la de dentro.

El ruido del percentil efectivo. Con 62 ganadoras en `oos1`, aunque la distribución fuera idéntica, el porcentaje que cae bajo el p90 verdadero baila con desviación √(0,9 × 0,1 / 62) = 3,8 puntos; y la propia X del `build` tiene su error (2.5). La tolerancia de 5 puntos es poco más de una desviación.

Nulo calculado por mí (20.000 tiradas por estrategia, semilla 24, `s4_mine.py`): saco n ganadoras «de dentro» y m «de fuera» **de la misma distribución**, con los n y m reales de cada estrategia, y cuento cuántas veces la etiqueta dice «se transfiere». El resultado no depende de la forma de la distribución.

percentil	probabilidad de «se transfiere» siendo las dos ventanas iguales (rango entre las diez)	observado en `oos1`
80	41 % – 65 %	4 de 10
90	56 % – 78 % (suma: 6,8 de 10 esperadas)	5 de 10
95	80 % – 93 %	7 de 10

Con estas muestras, **«no se transfiere» sale entre el 22 % y el 44 % de las veces al p90 sin que haya cambiado nada**. Cinco de diez es compatible con que todas se transfieran.

Valores reales al p90 (pretendido: 90):

estrategia	ganadoras `oos1`	efectivo `oos1`	cociente de medianas `oos1`	p de KS	ganadoras `oos2` (guardado)	efectivo `oos2` (guardado)
10.11.484	62	88,7	1,00	0,37	60	90,0
13.3.390	31	74,2	1,75	0,06	27	88,9
14.3.479	44	77,3	1,27	0,34	49	85,7
15.9.352	86	75,6	1,41	0,07	63	77,8
16.5.352	91	97,8	0,95	0,43	71	90,1
18.5.355	57	94,7	1,23	0,19	39	97,4
19.2.363	35	74,3	1,19	0,40	78	80,8
3.11.426	59	88,1	1,22	0,61	40	90,0
3.9.284	78	92,3	1,06	0,63	60	95,0
4.13.338	72	87,5	1,08	1,00	64	87,5

Ningún KS baja de 0,05. El estudio **no recalcula** la X fuera de muestra, y ése es el diseño correcto: recalcularla sería ajustarla al tramo de comprobación.

2.10 Meseta y borde

La rejilla de la pasada 2 prueba, alrededor de cada X, cinco valores: X × {0,8 · 0,9 · 1 · 1,1 · 1,2} (`grid.py:21-24`; `grid.band: 0.20`, `grid.steps: 2`). La idea: si el resultado apenas cambia al mover X un 20 %, la X está en una **meseta** y su error de muestreo (2.5) importa poco; si cambia mucho, está en un **borde**.

«Cambia mucho» se mide con una puntuación (`stability.py:86-105`), la del dueño del 2026-09-26:

> puntuación = 0,40 × PF / PF original + 0,30 × neto / neto original + 0,30 × DD original / DD

Cada término es un cociente contra la original sin stop (el drawdown, invertido), así que **1,0 significa «igual que sin stop»**. Ejemplo: PF 1,54 contra 1,48, neto 20.682 contra 20.297, DD 8.188 contra 6.605 → 0,40 × 1,04 + 0,30 × 1,02 + 0,30 × 0,81 = **0,96**.

La etiqueta (`stability.py:108-134`): se mira cuánto se aparta la puntuación de cada vecino de la del centro, como fracción de la del centro. Si ningún vecino se aparta más del 10 % (`shape.tolerance: 0.10`) → **«meseta»**. Si se apartan los de abajo (X más corta) → «borde al apretar»; los de arriba → «borde al aflojar»; los dos → «borde por los dos lados». Si la original no gana en esa ventana → «sin referencia».

Dos avisos sobre la puntuación. El término del neto es un cociente: cuando la original gana poco, se dispara (`19.2.363` en `build`: neto 1.687 → 4.049, puntuación 1,50). Y el DD es de cierres (5.6).

---

3. Lo que hace o calcula, pieza a pieza

3.1 La X de cada percentil y su intervalo

`threshold.x_values` (`threshold.py:31-48`). Por percentil: `x`, `low`, `high`, `width` = (high − low)/x, `unreliable` = width > 0,5. Pasan a `verdict.csv` como `x_p90`, `low_p90`, `high_p90`, `unreliable_p90` (`one.py:51-59`). Tabla real en 1.5.

3.2 La zona de cada X

`noreturn.curve` y `noreturn.zone_of` → `zone_p90`. Detalle de implementación: `zone_of` lee la fila de la curva **en el escalón de 0,25 inmediatamente por debajo** de X (`noreturn.py:49-50`). Con X = 2,69 lee la fila de 2,50 («ruido») y no la de 2,75 («sin retorno»).

3.3 La transferencia

`transfer.effective` → `effective_oos1_p90`, `effective_oos2_p90`. La marca `transfers` (sí/no/—) se calcula y se pinta en la pestaña, pero **no llega a `verdict.csv`** (`one.py:51-59` sólo guarda el efectivo). `transfer.shape` → D y p de KS, Anderson-Darling y cociente de medianas, sólo en la pestaña.

3.4 El lote: `stopgrid.csv`, `sqx.variants.stopgrid` y el injerto

- `grid.rows` (`grid.py:8-24`) escribe `stopgrid.csv`: por estrategia, percentil y paso (−2 a +2), `x = X × (1 + 0,20 × paso / 2)`, a cuatro decimales. Para `10.11.484`, p90: 1,7687 · 1,9897 · **2,2108** · 2,4319 · 2,6530.

- `stopgrid.plan` (`stopgrid.py:18-46`) decide los ficheros de cada madre: `reference` (la original), `probe` (X = 1.000) y una fila `grid` por X. Los nombra `S<madre>V<n>` (la variante p90, paso 0 de la primera madre es `S00V014`).
- `stopgrid.write` (`stopgrid.py:49-68`) los escribe. La referencia sólo cambia de nombre. A las demás les aplica el injerto.
- **El injerto** (`stoploss.graft`, `stoploss.py:31-62`): dos sustituciones de texto en el `strategy_Portfolio.xml`. En cada orden de entrada, la fórmula «sin stop» (`SLPT.None`) se cambia por `ATRBasedValue` con el multiplicador en una variable `StopLossCoef1` y el periodo 20 como literal; y se declara esa variable como parámetro externo. El resto del fichero queda byte a byte igual.

Las X de los cuatro percentiles se solapan: en `10.11.484` las veinte X van de 1,32 a 3,22 y forman un barrido casi continuo. Es, de hecho, una rejilla de veinte stops con su resultado en las tres ventanas, que nada en el código usa para elegir.

3.5 Las dos pruebas

La del injerto (`proofs.graft`, `proofs.py:12-40`). La sonda con stop a 1.000 ATR nunca salta, así que tiene que dar las mismas operaciones que la original: mismas cuatro columnas de hora y precio, y P/L a menos de 0,01 USD (`proof.pl_tolerance`). Si no, el injerto cambió algo más que el stop. → `graft_identical`. **En la corrida: idéntica en las diez y en las tres ventanas** (recalculado para `build` y `oos1`: 0 operaciones distintas, diferencia máxima 0,00).

La del ATR (`proofs.atr`, `proofs.py:43-74`). Para cada operación cerrada por stop: |cierre − entrada| / (X × ATR), con el ATR de tres velas candidatas (la de entrada, la anterior, dos antes). En la vela correcta ese cociente debería ser 1 más un pelo de deslizamiento para todas las operaciones; la prueba pasa si del percentil 5 al 95 se separa 0,02 o menos (`proof.atr_spread`). → `atr_matches`.

En la corrida: `atr_matches` es `True` en 2 de 10 (guardado; incluye `oos2`). Recalculado, en `build` pasa en 2 de 10 y en `oos1` en 7 de 10. Por qué (calculado por mí, `s6_atrproof.py`, las 15.512 salidas por stop de toda la rejilla en `build` y 9.810 en `oos1`):

	`build`	**`oos1`**
cociente, percentiles 5 / 50 / 95	1,004 / 1,013 / 1,065	1,005 / 1,013 / 1,037
el mismo quitando el deslizamiento fijo de la tarea (1,06 y 1,66 puntos)	1,000 / 1,000 / 1,036	1,000 / 1,000 / 1,000
stops ejecutados en una apertura con hueco, ya más allá del nivel	853 (5,5 %)	166 (1,7 %)
sin hueco y sin deslizamiento, percentiles 5 / 50 / 95	0,9998 / 1,0000 / 1,0000	0,9999 / 1,0000 / 1,0000

El ATR está bien, exacto. La prueba falla por dos motivos que no tienen que ver con él: el deslizamiento es fijo en puntos y los stops miden entre 33 y 249 puntos, así que pesa entre el 0,4 % y el 3,2 % (más que la tolerancia); y los huecos. La tolerancia de 0,02 se midió en el oro (`config.yaml:50-52`). `atr_matches` exige las tres ventanas (`stability.py:141`); el fallo se concentra en `build`, donde hay más huecos (5,5 % frente a 1,7 %) y stops más estrechos en puntos. Guardado en las demás corridas: `atr_matches` es `True` en 0 de 10 en las cuatro `Test_NIKKEI225_M30_simpleL_*` y en 3 de 3 en USDJPY y USA500.

Los huecos son el dato importante para fondeo. Cuando la vela abre por debajo del nivel del stop, SQX ejecuta en la apertura menos el deslizamiento (comprobado: 100 % de los casos). El exceso más allá del nivel, en ATR: mediana 0,89, percentil 95 de 3,34 y máximo 12,1 en `build`; 0,55 / 2,50 / 3,05 en `oos1`. Por

estrategia, entre el 2 % y el 10 % de los stops del `build`. (Estas cifras son de toda la rejilla de 200 variantes, donde la misma operación se cuenta hasta veinte veces; sobre las diez variantes del p90 que se operan, véase el punto 4 de 0.)

3.6 Lo que cuesta cada X

`stability.stats` y `stability.against` (`stability.py:10-44`), por variante de la rejilla y por ventana, casando las operaciones con las de la original por hora de entrada:

salida	qué es
`n`, `net`, `pf`, `maxdd`, `worst`	operaciones, neto, profit factor, drawdown máximo **de la curva de cierres** y peor operación, con stop
`stopped`, `stopped_pct`	cuántas cerró el stop
`killed`	de ésas, cuántas eran ganadoras en la original
`saved`	pérdida evitada en las que la original perdía
`given_up`	ganancia perdida en las que la original ganaba
`new`	entradas que la original no tomó
`…_original`	los mismos cuatro números de la original

Resultado real al p90, centro de la rejilla (recalculado; cada celda es original → con stop):

estrategia	X (USD nominal)	ops `build`	paradas	neto `build`	peor op. `build`	neto `oos1`	PF `oos1`	peor op. `oos1`
10.11.484	2,21 (553)	154 → 159	38 %	20.297 → 20.682	−5.652 → −815	14.769 → 9.819	1,62 → 1,36	−1.933 → −587
13.3.390	3,34 (835)	143 → 179	37 %	14.822 → 12.507	−6.019 → −1.338	5.342 → **−1.825**	1,33 → 0,93	−2.116 → −966
14.3.479	1,18 (295)	189 → 206	31 %	13.239 → 13.375	−1.459 → −368	4.690 → 198	1,30 → 1,01	−1.382 → −313
15.9.352	2,27 (568)	224 → 289	41 %	2.944 → 33	−2.801 → −1.421	5.626 → 1.872	1,11 → 1,03	−2.289 → −1.133
16.5.352	4,59 (1.148)	304 → 317	30 %	10.406 → **−6.593**	−4.594 → −4.180	9.005 → 12.060	1,11 → 1,16	−4.092 → −1.414
18.5.355	4,20 (1.049)	161 → 162	29 %	42.168 → 29.340	−4.013 → −1.975	5.392 → 7.752	1,13 → 1,20	−2.440 → −1.140
19.2.363	1,26 (314)	161 → 174	36 %	1.687 → 4.049	−2.080 → −1.055	2.311 → **−2.024**	1,26 → 0,85	−1.085 → −331
3.11.426	2,35 (589)	135 → 140	36 %	22.508 → 24.145	−3.502 → −1.128	19.670 → 17.199	1,93 → 1,82	−2.403 → −869
3.9.284	2,55 (637)	222 → 227	23 %	32.990 → 30.268	−3.405 → −732	10.197 → 5.351	1,26 → 1,14	−2.719 → −1.056
4.13.338	2,75 (688)	198 → 204	36 %	53.725 → 55.385	−4.399 → −1.436	22.721 → 17.157	1,57 → 1,44	−3.219 → −735

Por percentil, las diez juntas (calculado por mí desde las mismas filas):

ventana	percentil	paradas (mediana)	ganadoras muertas	nuevas	neto con stop / original	ganan menos	pasan a perder	peor op. con stop / original (mediana)	DD de cierres sube en
`build`	80	47 %	197	321	149.596 / 214.787 (−30 %)	8 de 10	0	0,29	5
`build`	85	41 %	151	250	177.190 (−18 %)	7	0	0,29	5
`build`	**90**	36 %	102	196	183.190 (−15 %)	5	1	0,32	5
`build`	95	25 %	53	122	202.309 (−6 %)	5	1	0,37	6
`oos1`	80	46 %	135	194	59.104 / 99.724 (−41 %)	8	3	0,24	5
`oos1`	85	40 %	109	169	73.977 (−26 %)	8	3	0,35	5
`oos1`	**90**	38 %	80	131	67.559 (−32 %)	8	2	0,35	6
`oos1`	95	26 %	44	79	75.737 (−24 %)	10	2	0,38	5

Lo que el stop compra es la última columna de «peor operación»: queda en un tercio. Lo que cuesta es entre un 6 % y un 41 % del neto. Y el drawdown de cierres no mejora.

La peor operación con stop frente a la pérdida nominal del stop (X × 250 USD; calculado por mí): al p90, en `build`, entre 1,15 y 3,64 veces (mediana 1,90; máximo de 688 stops); en `oos1`, entre 1,05 y 1,99 (mediana 1,12). El caso extremo es `16.5.352`: stop nominal de 1.148 USD, peor operación con stop −4.180. Son los huecos de 3.5.

3.7 La forma

`stability.shape` → `shape_<ventana>_p<percentil>`. En la corrida (guardado, 120 etiquetas = 10 × 4 × 3): borde por los dos lados 47, borde al apretar 22, borde al aflojar 22, **meseta 21**, sin referencia 8. Al p90: meseta en 3 de 10 en `build`, 2 de 10 en `oos1` y 2 de 10 en `oos2`.

Ejemplo, `10.11.484`, p90, `build`: puntuaciones 0,97 · 1,03 · **0,96** · 1,14 · 1,07 a lo largo de la rejilla. El vecino de arriba (1,14) se aparta un 18 % del centro → «borde al aflojar». Lo que ha pasado entre 2,21 y 2,43: el stop deja de matar tres ganadoras (de 9 a 6) y el neto sube de 20.682 a 23.904. Tres operaciones.

3.8 Lo que queda escrito

fichero	qué lleva
`<lote>/pass1/` y `pass2/` → `sqx/S00V000.sqx…`	los `.sqx` fabricados; la carpeta se vacía al refabricar
`…/manifest.parquet`	qué fichero es qué madre, qué percentil, qué paso y qué X
`…/ran.json`, `retest_<tramo>.csv`	lo que devolvió el retest
`…/estudios/atrCalculator/estrategias/<madre>.json` y `.html`	lo que pinta la ventana
`…/estudios/atrCalculator/stopgrid.csv`	las X de la rejilla
`…/estudios/atrCalculator/verdict.csv` + `manifest.json`	una fila por madre: ganadoras por tramo, por percentil `x`, `low`, `high`, `unreliable`, `zone`, `effective_oos1/2`; `graft_identical`, `atr_matches`, y `shape_<tramo>_p<percentil>`
`AlgoData/raw/<proyecto>/<pata>/<fecha>/stopgrid/trades.parquet`	las operaciones de cada pata; la pasada 2 pisa a la 1 si son del mismo día
`AlgoData/ledger/<activo>_<tf>_<familia>.jsonl`	una fila por tramo y pasada, `criterion: atrCalculator/stop`

3.9 Lo que sale hacia el paso 25

`pipeline/autopilot/traded.py`. Por madre:

1. Busca el lote más reciente que llegó a la pasada 2 (`traded.py:23-27`).
2. En su `manifest.parquet`, la fila con `percentile == 90` (el valor del ledger) y `step == 0` (`traded.py:47`).
3. Ésa es **la versión que se opera**: el `.sqx` injertado con `StopLossCoef1` = X del p90 a cuatro decimales.

Qué cambia en la estrategia respecto a la madre:

- En cada orden de entrada, un stop `X × ATR(20)` de la vela cerrada antes de la entrada, fijado al entrar (no se mueve después).
- Un parámetro externo nuevo, `StopLossCoef1`, que el EA de MT5 enseñará como entrada.
- Otro nombre (`Strategy 10.11.484 S00V014`) y otra identidad.
- Nada más: condiciones, salidas, tamaño y horarios, intactos.

Qué **no** mira `traded.py` al tomarla: ni `graft_identical`, ni `atr_matches`, ni `unreliable_p90`, ni la zona, ni la transferencia, ni la forma. Una madre sin fila p90 en el lote (por ejemplo, si el estudio se corrió con otros percentiles) queda fuera con el aviso «SIN stop, fuera».

Después (`traded.py:72-107`, `110-129`): las operaciones de esas variantes en `build` y `oos1` se archivan como la cosecha `Stop_p90`, y sobre ella corre el **edge por coste** del paso 25, que sólo lee `oos1` (`studies/readings/edgeCost/costs.py:64`). `finalists.py:15-42` escribe `finalistas.json` para el paso 26 con el fichero, la X y el percentil de cada una. En la corrida: diez finalistas, las diez al p90, X de 1,18 a 4,59.

3.10 Relación con la criba del MAE profundo (paso 8)

`studies/screening/maeDepth/` usa el mismo motor (`engines/market/excursion.py`), así que mide el mismo MAE en ATR, operación por operación. Pregunta otra cosa y antes: sobre la población entera en el paso 8, **cuánto del neto de cada estrategia viene de operaciones que fueron 3 o 4 ATR en contra y volvieron** (`measures.py:65-73`), y cuánto quedaría si todas ésas se hubieran cerrado en −(k + deslizamiento) ATR. Es el aviso temprano de lo que el paso 24 mide como `given_up`: una estrategia que vive de operaciones recuperadas desde muy hondo es una a la que el stop le va a costar caro.

Dos diferencias a tener presentes. Su contrafactual es una **cota** hecha en Python, no un backtest: no ve las operaciones nuevas (2.7) ni los huecos (3.5). Y su umbral de marca (`verdict.min_cuota_rec`) está en `null`: hoy mide y no marca a nadie.

---

4. El criterio cargado y por qué

4.1 En el autopiloto: ninguno

`criteria.yaml` no tiene entrada `"24"`. `facts.py` no extrae ningún hecho del paso. `tail.py:16` lo trata como «un lote retesteado en las tres patas WFC» y nada más. El paso 24 no puede dar Pasa, Limbo ni Falla.

4.2 El único umbral que decide

```
- key: portfolio.funded.stop_percentile
  value: 90
  set_by: dueño        set_on: 2026-09-30
  why: el stop con el que se opera es SL = X·ATR(20) del paso 24; X = p90 del MAE/ATR de las
       ganadoras de build, la misma regla para todas, fijada antes de mirar ninguna cartera
```

(`ledger/thresholds.yaml:1133-1138`). Lo leen `traded.py:14` (paso 25) y la construcción de carteras (`portfolio/common/construct/config.yaml:40`). Los percentiles que **calcula** el estudio (`config.yaml:10`) son otra lista independiente: si no incluye el 90, el paso 25 se queda sin versión que operar.

4.3 Las negativas del código

qué se niega	dónde
leer un tramo reservado sin permiso (sólo bajo agente autónomo)	`spend.py:9-21`, `report.py:45`, `closing.py:115-116`
un percentil fuera de (0, 100), o ninguno	`inputs.py:52-54`
un databank sin ganadoras en `build` (un retest OOS)	`one.py:33-37`
injertar en una estrategia que ya declara `StopLossCoef1`	`stoploss.py:46-47`
un injerto que no pone exactamente un stop por orden de entrada	`stoploss.py:54-57`
un lote cuyos ficheros no son el plan	`stopgrid.py:88-90`
leer un export que no es el de la pasada 2 de ese lote	`traded.py:66-68`
el paso 25 sin ninguna madre con stop	`traded.py:120-122`

4.4 Las etiquetas que describen y no deciden

etiqueta	umbral	de dónde sale el umbral
«poco fiable»	intervalo > 50 % de X	`bootstrap.wide`, sin fecha ni autor en el fichero
«sin retorno»	≤ 10 % se recupera y media final < −x; mínimo 10 operaciones	`noreturn.recover_max`, `min_reached`
«se transfiere»	a ≤ 5 puntos de percentil; mínimo 30 ganadoras	`transfer.tolerance`, `min_winners`
«meseta»	puntuación a ≤ 10 % de la del centro	`shape.tolerance`, «provisional: el dueño fijará la suya»
prueba del injerto	P/L a ≤ 0,01 USD	`proof.pl_tolerance`
prueba del ATR	p95 − p5 ≤ 0,02	`proof.atr_spread`, medido en XAUUSD M30 el 2026-09-26

Ninguno de estos seis está en `ledger/thresholds.yaml`: viven sólo en el `config.yaml` del módulo.

4.5 El resultado sobre la corrida real

	resultado
estrategias que entran / que salen del paso	10 / 10
prueba del injerto	10 de 10 idénticas
prueba del ATR	2 de 10 (el ATR es correcto en las diez; ver 3.5)
X del p90 «poco fiable»	2 de 10 (`13.3.390`, `14.3.479`)
zona de la X del p90	«ruido» en las 10
«se transfiere» al p90	5 de 10 en `oos1`
«meseta» al p90	3 de 10 en `build`, 2 de 10 en `oos1`, 2 de 10 en `oos2` (guardado)
al paso 25	las 10, todas con la X del p90

---

5. Lo que no cuadra o no está medido

5.1 El manual promete un stop que «salta poco y cambia poco»

`54-atr-calculator.md:23-24`: «el stop se pone sin cambiar la estrategia, y lo esperable es que salte poco y cambie poco». Medido al p90: salta en el 23-41 % de las operaciones, añade un 10 % de operaciones nuevas y en `oos1` se lleva el 32 % del neto de las diez (3.6). El propio ejemplo del manual (oro) ya traía un 34 % de paradas al p90.

5.2 La versión con stop no vuelve a pasar por nada

`WORKFLOW.md` («El paso 24»): «Después, si hace falta, se repite alguno de los tests anteriores sobre la versión con stop». El autopiloto sólo le corre el paso 25 (edge por coste, que lee sólo `oos1`: `costs.py:64`). Dos de las diez pierden en `oos1` con el stop y siguen como finalistas. Qué tests repetir es decisión del dueño (el manual lo dice: «Qué tests se repiten sobre la versión con stop lo decide el dueño»); hoy la respuesta cargada es «ninguno».

5.3 «Si alguna prueba dice no, lo demás no vale» — y sigue

`54-atr-calculator.md:138-140`. `traded.py` no lee `graft_identical` ni `atr_matches`. Con la prueba del ATR en «no» en ocho de diez, el paso 25 tomó las diez. En este caso el «no» es un falso aviso (5.4), pero un injerto roto de verdad pasaría igual.

5.4 La tolerancia de la prueba del ATR es de un activo

`proof.atr_spread: 0.02` se midió en el oro (`config.yaml:50-52`). En el Nikkei falla con el ATR exacto, por el deslizamiento fijo y los huecos (3.5). En las cuatro corridas `Test_NIKKEI225_M30_simpleL_*` falla en diez de diez. La prueba, como está, no separa «ATR equivocado» de «activo con huecos».

5.5 El glosario y el motor dicen «camino M1»; las patas corren a M30

`view.py:10-11` («medido por SQX sobre el camino M1») y `excursion.py:39`. Las tareas WFC llevan `testPrecision="1"` (timeframe elegido) desde la decisión del 2026-10-02. El número no cambia en estas estrategias (2.2), pero la frase ya no describe lo que corre, y dejaría de ser inocua con entradas dentro de la vela o con cortas.

5.6 El drawdown es de cierres

`stability.py:17` calcula el drawdown sobre la suma acumulada de los P/L cerrados. Entra con un 30 % en la puntuación de la forma y sale en la tabla de coste. No hay ninguna medida de drawdown **flotante** de la versión con stop, que es la que cuenta en fondeo; `knowhow/export/mae-mfe-from-m1.md` anota que el paso 24 no guarda la curva diaria de mínimos de las variantes.

5.7 El stop no acota la pérdida y nada lo dice

La tabla de coste enseña la peor operación, pero no la compara con X × ATR. No hay ninguna salida que diga «el x % de los stops se ejecutó con hueco, hasta n ATR más allá». Lo medido está en 3.5 y 3.6 y es mío.

5.8 «No se transfiere» no sale de la pestaña, y la tolerancia está dentro del ruido

El README (`README.md:62`) dice que si la X no se transfiere «the report says so and stops», es decir, lo dice y no recalcula la X, que es lo que hace (`transfer.py:24-31`). Lo que falta es que la marca no llega a `verdict.csv`, porque `one.py:56-58` sólo guarda `effective_oos1/2` (3.3). Aparte, con estas muestras la marca salta sola entre el 22 % y el 44 % de las veces al p90 (2.9).

5.9 La rejilla cuesta y no alimenta ninguna decisión

Veinte variantes por madre en tres ventanas, y una segunda mirada a `oos2`, para producir una etiqueta que `traded.py` no lee. Además, la etiqueta es frágil: en `10.11.484` la diferencia entre «meseta» y «borde» son tres operaciones (3.7).

5.10 Cuatro percentiles a la vista con el neto al lado

El diseño «no elige», pero pone delante cuatro candidatas (veinte, contando la rejilla) con su neto en `build`, `oos1` y `oos2`. Mientras el 90 siga fijo, no hay problema. Si se cambia tras mirar, el ledger no lo registra como búsqueda (2.6).

5.11 El paso corre sobre toda la población y con los mercados adicionales puestos

El manual (`:19-20`): «sobre una estrategia que ya pasó la secuencia entera… una, no una población». `closing.py:101` lo corre sobre todas las madres que quedan. Y las patas llevan los once mercados adicionales: el 92 % de las operaciones retesteadas en `build` se descarta (1.2).

5.12 El estado del paso en `WORKFLOW.md` está atrasado

La fila 24 dice «construido y probado en SQX sobre una estrategia de desarrollo; falta una superviviente real». En disco hay nueve lotes con pasada 2 de ocho proyectos (2026-09-30 a 2026-10-06).

5.13 Detalles menores

- `zone_of` lee la fila del escalón de 0,25 inmediatamente por debajo de X (3.2).
- El capítulo del manual se titula «# 49.» y el fichero es el 54.
- El ejemplo del manual usa un feed con el nombre anterior al 2026-10-01 (`XAUUSD_DukasM1_Infinox`).

5.14 Lo que no está medido

- El MAE y el stop con **ticks reales y spread variable**, y en **cortas**.
- El **drawdown flotante** y la **peor pérdida intradía** de la versión con stop.
- Si la X de las ganadoras del `build` sale **corta por selección** (2.7): haría falta una población sin ventaja, o comparar con estrategias no elegidas por su `build`.
- El **intervalo de X con operaciones dependientes** (bootstrap por bloques).
- Qué hace el stop en los pasos 8 a 20: ninguno se ha corrido sobre una versión con stop.
- Lo que pide MT5 (`StopsLevel`, distancia mínima del bróker): fuera del paso por decisión del dueño del 2026-09-26. Con stops de 295 USD nominales (1,18 ATR, unos 45 puntos en mediana) no está comprobado que ningún bróker lo rechace.
- El **flotante conjunto** de las hasta 8 posiciones abiertas a la vez, y la hora real del stop (sellada en la apertura de la vela, hasta 29 minutos antes), sin la cual no se puede repartir una pérdida entre dos días de la firma.

5.15 El coste del stop viene de las ganadoras grandes que mata

Descomposición del cambio de neto al p90, suma de las diez (calculado por el revisor, con el código del estudio sobre los exports de la pasada 2, sólo `build` y `oos1`): en `build`, −31.597 USD = ganadoras muertas −152.489 (102 operaciones, media −1.495 USD cada una, el 71 % del neto original de 214.787) + perdedoras paradas +98.701 (503 paradas: 272 mejoran, 231 empeoran, saldo positivo) + operaciones desaparecidas +7.950 + nuevas +14.241. En `oos1`, −32.165 = −96.920 + 53.490 + 2.398 + 8.867. El coste entero viene del 10 % (13 % en `oos1`) de ganadoras que el stop se lleva, que son las más grandes (las que van hondo y luego rinden). Cortar perdedoras ayuda en neto.

5.16 Las cuentas de fondeo no exigen un stop

El catálogo del proyecto dice, de las dos firmas en uso, FTMO «No stop loss required» (`AlgoData/funding/rules/ftmo.yaml:164`) y Hantec «Stop loss not mandatory, strongly recommended» (`hantec.yaml:258`). La frase «el stop loss que MT5 exige» viene de `WORKFLOW.md:102`. El stop es una decisión de riesgo, y el paso cuesta el 32 % del neto de `oos1`; lo que las firmas exigen es no pasar la pérdida diaria ni la total, contando el flotante (`ftmo.yaml:44-47`).

5.17 No hay ninguna cifra de las diez juntas

El dossier mide la pérdida por operación y por estrategia. Lo que hay de las diez juntas (calculado por el revisor, versiones con stop al p90, tamaño del backtest de 250 USD por ATR sobre 100.000; por cierres, sin flotante): las diez son largas del mismo índice; llegan a estar abiertas a la vez 8 en `build` y 7 en `oos1`; hay 7 días en `build` y 4 en `oos1` en que el stop saltó en cinco o más el mismo día (máximo 6). La suma de los diez stops nominales es 26,70 ATR = 6.676 USD. Peor día por cierres con stop: −5.311 USD en `build` (2017-11-09) y −4.303 en `oos1` (2022-03-23), el 5,3 % y el 4,3 % del capital inicial; sin stop eran −12.204 y −8.214. El flotante de las que siguen abiertas ese día no está contado, y FTMO cuenta cerrado más flotante (`ftmo.yaml:44-47`). `portfolio/common/construct/config.yaml:44` tiene `open_risk_cap: null`.

5.18 De dónde salen los 250 USD por ATR y qué implica

`assets/_build.yaml:60-63`: `ATRRiskBasedSizingFixedRisk`, `Amount` 1000, `ATRMult` 4, `ATRPeriod` 20; 1.000 ÷ 4 = 250 USD por ATR (calculado por el revisor: tamaño × valor del punto × ATR queda entre 249,9 y 250,0 en las 1.891 operaciones de `build`). El tamaño supone 4 ATR de riesgo: con X > 4 la pérdida nominal del stop supera los 1.000 USD con que se dimensiona (`16.5.352`: X 4,59 → 1.148 USD; `18.5.355`: X 4,20 → 1.049 USD) y con X < 4 es menor (`10.11.484`: 553 USD). El lote es inverso al ATR de las últimas diez horas: el ATR en las entradas de `build` va de mediana 38 a máximo 237 puntos (6,2 veces) y el lote baja en proporción, pero la exposición es máxima justo tras las horas más quietas, y el hueco de apertura no se mide en ese ATR. Tres de los cinco mayores excesos sobre el stop de `build` entraron con el ATR en el 4 % más bajo (calculado por el revisor): `16.5.352` 2017-08-08, ATR 16,3 puntos (percentil 1), 242,7 lotes contra 103 de mediana, −4.180 USD; `16.5.352` 2012-10-04, percentil 4, 179 lotes, −2.494; `15.9.352` 2017-03-07, percentil 1, 213 lotes, −1.421. En el conjunto no hay sesgo: el ATR mediano de los stops con hueco es el mismo que el de todos los stops (percentil 32). La cartera recalcula el tamaño sobre X × ATR nominal (`risk.py:49-50`).

5.19 La peor operación observada es un suelo

Sobre las diez variantes del p90 (calculado por el revisor): en `build`, 43 stops con hueco de 688 (6,2 %), en 37 fechas; exceso mediano 0,89 ATR, percentil 95 de 4,92, máximo 11,97. En `oos1`: 10 de 450 (2,2 %), en 8 fechas; máximo 2,15. La cola son tres operaciones de una sola estrategia (`16.5.352`: 11,97 · 7,41 · 5,07 ATR); sin ellas el máximo de `build` es 3,61. Un máximo que depende de tres operaciones y que pasa de 12 a 2 entre ventanas no es una cota: es la peor de las ~1.100 que se han visto. Además, a `testPrecision=1` SQX llena todo stop tocado dentro de la vela exactamente en el nivel menos el deslizamiento fijo (la tabla de 3.5: sin hueco, cociente 0,9998 / 1,0000 / 1,0000); sólo existen los huecos entre velas M30, y el deslizamiento real del stop dentro de la vela (noticias, apertura de Tokio) vale 1,06 / 1,66 puntos por decreto.

5.20 El percentil no se fijó antes de ver resultados con stop

La regla de 2.6 se presentaba como escrita antes de ver ningún resultado con stop. Hay corridas del paso anteriores o del mismo día que el umbral: el ledger (`criterion: atrCalculator/stop`) tiene la corrida de oro del 2026-09-26 y las de USDJPY donchian M30 (2026-09-28/29/30) y H1 (2026-09-30, 13:22, 13:28, 19:06); el capítulo del manual (`54-atr-calculator.md:188-198`, del 2026-09-29) ya trae neto, PF y peor operación al p80/90/95 del oro. El percentil se fijó el 2026-09-30 (`ledger/thresholds.yaml:1133-1138`), cuyo `why` dice «antes de mirar ninguna cartera», no «antes de mirar el coste del stop». No consta si el 90 se eligió mirando esas tablas.

---

6. Cierre

6.1 Las decisiones que te quedan

#	decisión	qué ganas	qué cuesta
1	**Mantener el percentil 90 o moverlo**, y si se mueve, hacerlo sin mirar la tabla de coste	Al p95 el stop cuesta menos neto (−6 % en `build`, −24 % en `oos1`, frente a −15 % y −32 %) y mata la mitad de ganadoras. En `oos1` el orden no se sostiene (el p85 cuesta −26 %, menos que el p90): con diez estrategias, la diferencia entre p90 y p95 fuera de muestra no se distingue del ruido	La X del p95 es «poco fiable» en 6 de 10 (2 de 10 al p90) y la peor operación queda algo más grande (0,37 frente a 0,32 de la original). Moverlo tras ver estas cifras ya es elegir por resultado
2	**Qué tests repite la versión con stop** antes de ir a MT5	Saber si la estrategia que se opera sigue teniendo lo que tenía la que pasó la cadena; hoy 2 de 10 pierden en `oos1` con stop	CPU, y si el test lee `oos2`, otra mirada. En estrategias: con «neto positivo en `oos1`» como única vara caerían 2 de 10
3	**Si las pruebas bloquean** (injerto, y ATR con una tolerancia que valga para cada activo)	Un injerto roto no llegaría a MT5	Ninguna estrategia con la del injerto (10 de 10 pasan). La del ATR, tal como está, tumbaría 8 de 10 sin motivo: antes hay que rehacerla
4	**Con qué número se dimensiona en fondeo**: X × ATR o la peor operación observada con stop	La peor con stop es 1,15-3,6 veces la nominal en `build` (máximo de 688 stops; en `oos1`, 1,05-2,0). Es un suelo, no un techo (5.19); y el stop no es un requisito de la firma (5.16)	Tamaños más pequeños
5	**Medir el flotante de la versión con stop** (drawdown flotante, peor día)	Es la medida que decide una cuenta de fondeo; hoy sólo hay cierres	Guardar la curva de mínimos de las variantes, que el paso no guarda
6	**La rejilla de estabilidad: se usa o se quita**	Si se quita: una pasada menos, una mirada menos a `oos2`, 200 ficheros (600 retests de ventana) menos por cada diez madres	Se pierde la tabla de coste por X vecina, que es lo que permite ver que el stop más flojo cuesta menos
7	**Las etiquetas «se transfiere» y «poco fiable»: decidir si significan algo**	Dejar de leer como señal lo que es ruido de muestra	Para que distingan haría falta más ganadoras por ventana de las que hay (31-91 en `oos1`)
8	**Quitar los mercados adicionales de las patas durante este paso**	El 92 % del retest del `build` se tira	Volver a ponerlos para quien los use después
9	**El umbral de la criba del MAE profundo (paso 8)**, hoy sin fijar	Avisaría en el paso 8 de las estrategias a las que el stop les va a costar más	Su coste en estrategias no está medido sobre una población

6.2 El límite que no cambia

El paso hace bien lo que promete: saca una X por regla, igual para todas, sin mirar el resultado con stop; el injerto es limpio (diez de diez idénticas) y el ATR es exactamente el de SQX. Lo que no puede hacer es que el stop sea gratis. Un stop a X ATR convierte una estrategia sin stop en otra distinta, con un 10 % de operaciones que nadie ha examinado, y en un mercado con huecos no garantiza la pérdida máxima. La X se lee del MAE; **el precio del stop, y si la estrategia lo aguanta, sólo lo dice el retest** — y hoy ese retest se calcula, se enseña y no lo juzga nadie.